

8. Introduction

Every developer eventually hits the same wall. You have a table with thousands of rows, and someone asks a deceptively simple question: *"How many orders did we get per city?"* or *"What's the average score in each class?"* You could export the data and count by hand, but that isn't engineering — that's data entry.

This is exactly where SQL earns its keep. Two clauses, `GROUP BY` and `HAVING`, quietly turn ten thousand raw rows into five clean answers. They're among the most-used tools in any backend engineer's daily work, and they show up constantly in coding interviews for data-heavy roles. Interviewers love them because they test something deeper than syntax: **do you understand how a database actually processes a query, step by step?**

If you've ever written `WHERE COUNT(*) > 100` and been rewarded with a cryptic error, this article is for you. By the end, you'll know how aggregate functions collapse rows, how grouping sorts data into buckets, why `WHERE` and `HAVING` are *not* interchangeable, and how to avoid the three mistakes that trip up nearly every beginner. We'll build the intuition first, then write clean queries, then look at the same idea in Python so it truly sticks.

9. Problem Overview

Here's the problem in plain terms.

You have a table where each row is one small fact — a single order, a single receipt, a single test result. That's great for detail, but useless for the big picture. Nobody wants to scroll through 10,000 orders. They want a **summary**: one number per category.

The task is to answer questions of the form *"give me one summary value for each group of related rows."* Total sales per city. Number of students per class. Average rating per product. The raw data holds the answer, but it's scattered across many rows that need to be **combined**.

To solve this cleanly, you need three concepts working together:

- **Aggregate functions** — tools like `COUNT`, `SUM`, and `AVG` that take *many* rows and return *one* number.
- **GROUP BY** — the instruction that decides *which* rows belong together in the same bucket before that number is computed.

- **HAVING** — a filter that runs *after* the buckets are formed, keeping only the groups you care about.

Master these three and you can reshape almost any dataset into a report.

10. Example

Imagine an `orders` table:

order_id	city	amount
1	Delhi	250
2	Mumbai	400
3	Delhi	150
4	Mumbai	100
5	Delhi	300
6	Pune	500

Question: How many orders and how much total revenue came from each city?

```
SELECT city,
       COUNT(*) AS order_count,
       SUM(amount) AS total_revenue
FROM orders
GROUP BY city;
```

Output:

city	order_count	total_revenue
Delhi	3	700
Mumbai	2	500
Pune	1	500

Read what happened. All the Delhi rows drifted into one pile: three orders totaling $250 + 150 + 300 = 700$. Mumbai formed its own pile with two orders summing to 500 . Pune had a single order. Six input rows collapsed into three summary rows — one per city. **That collapse is the entire point of GROUP BY.**

11. Intuition

Forget syntax for a moment and picture a shoebox full of shop receipts.

GROUP BY is you sorting those receipts into piles — one pile per store. You don't read every receipt anymore. Instead, you write a single total on the front of each pile. That's the mental model, and it's worth burning into memory: **GROUP BY makes piles, and you report one summary number per pile.**

Now, how does an experienced engineer *discover* this solution when facing a real dataset? They ask a sequence of questions:

1. "What's the unit of my answer?" If the answer is "one row per city," then city is what you group by.
2. "What number do I want on each pile?" A count? A sum? An average? That tells you which aggregate function to reach for.
3. "Do I want all the piles, or only some of them?" If only some — say, only cities with more than 100 orders — that filter has to happen *after* the piles exist. That's **HAVING**.

Notice the order of thinking mirrors the order the database executes: filter individual rows first, form the groups, compute the summary, then filter the groups. Once you internalize that pipeline, the confusing errors disappear, because you can always ask "*has the count been computed yet at this stage?*" If the answer is no, you can't filter on it.

That single question — "**does this value exist yet?**" — is the key that unlocks the whole topic.

12. Brute Force Solution

Idea: Before **GROUP BY** exists in your mind, the naive way to get per-city counts is to query each city separately.

```
SELECT COUNT(*) FROM orders WHERE city = 'Delhi';  
SELECT COUNT(*) FROM orders WHERE city = 'Mumbai';  
SELECT COUNT(*) FROM orders WHERE city = 'Pune';
```

Algorithm: 1. Find every distinct city (often a separate **SELECT DISTINCT city** query). 2. Loop over those cities in your application code. 3. Fire one filtered aggregate query per city. 4. Collect the results back together.

Advantages: - Conceptually simple — each query is a plain filtered count. - Easy to reason about one city at a time.

Disadvantages: - You must know the cities in advance, or run an extra query to discover them. - N round-trips to the database instead of one. On a large or remote database, network latency dominates.

- The database re-scans the table for every single city. - The logic lives in your application, not the database — more code, more bugs.

Time complexity: Roughly $O(k \times n)$ where k is the number of distinct cities and n is the number of rows, since each of the k queries can scan up to n rows. **Space complexity:** $O(k)$ to hold the results, plus the overhead of k separate query executions.

This works, but it fights against the database instead of using it. There's a far better way.

13. Optimal Solution

The optimal solution lets the database do the bucketing in a single pass. Here's the mental pipeline SQL follows, in order:

```
FROM      → read the rows
WHERE     → filter individual rows      (no aggregates allowed here)
GROUP BY  → sort surviving rows into piles
Aggregate → compute COUNT / SUM / AVG per pile
HAVING    → filter the piles            (aggregates allowed here)
SELECT    → choose columns to show
ORDER BY  → sort the final rows
```

Walk through a fully-loaded example: *"Show cities with more than 2 orders whose orders were placed in 2024, ranked by revenue."*

```
SELECT city,
       COUNT(*) AS order_count,
       SUM(amount) AS total_revenue
FROM orders
WHERE order_year = 2024      -- filters raw rows BEFORE grouping
GROUP BY city                -- makes one pile per city
HAVING COUNT(*) > 2          -- filters piles AFTER grouping
ORDER BY total_revenue DESC;
```

Two filters, two different jobs:

Clause	Runs when	Sees	Use it for
WHERE	Before grouping	Individual raw rows	Filtering rows (year, city)
HAVING	After grouping	Whole piles	Filtering groups (COUNT)

This is the "two-stage checkpoint": **WHERE is the bouncer at the door checking people; HAVING is the inspector checking the finished piles.**

The Golden Rule

There's one rule you cannot break:

Every column in `SELECT` must either appear in `GROUP BY` or be wrapped in an aggregate function.

Why? A Delhi pile holds three different `order_id` values. If you write `SELECT city, order_id ... GROUP BY city`, you're asking the database to print *one* `order_id` for a pile that has three — it can't choose, so it refuses. Either group by that column too, or aggregate it (`MAX(order_id)` , `COUNT(order_id)` , and so on). **Don't ask for a single value from a pile that holds many.**

14. Python Code

The same GROUP BY logic appears constantly in Python data work — both in raw SQL through `sqlite3` and in pandas. Here's a clean, production-quality version of both so the concept transfers across your stack.

```
import sqlite3
import pandas as pd

# --- Sample data ---
orders = pd.DataFrame(
    {
        "order_id": [1, 2, 3, 4, 5, 6],
        "city": ["Delhi", "Mumbai", "Delhi", "Mumbai", "Delhi", "Pune"],
        "amount": [250, 400, 150, 100, 300, 500],
    }
)

def summarize_with_sql(df: pd.DataFrame, min_orders: int = 1) -> pd.DataFrame:
    """Run a GROUP BY / HAVING query against an in-memory SQLite table."""
    with sqlite3.connect(":memory:") as conn:
        df.to_sql("orders", conn, index=False, if_exists="replace")
        query = """
            SELECT city,
                   COUNT(*) AS order_count,
                   SUM(amount) AS total_revenue
            FROM orders
            GROUP BY city
            HAVING COUNT(*) >= ?
            ORDER BY total_revenue DESC
        """
        return pd.read_sql_query(query, conn, params=(min_orders,))

def summarize_with_pandas(df: pd.DataFrame, min_orders: int = 1) -> pd.DataFrame:
    """The same result using the pandas equivalent of GROUP BY / HAVING."""
```

```

grouped = (
    df.groupby("city")                # GROUP BY city
    .agg(order_count=("order_id", "count"), # COUNT(*)
        total_revenue=("amount", "sum")) # SUM(amount)
    .reset_index()
)
# HAVING COUNT(*) >= min_orders
filtered = grouped[grouped["order_count"] >= min_orders]
return filtered.sort_values("total_revenue", ascending=False)

if __name__ == "__main__":
    expected = {"Delhi": (3, 700), "Mumbai": (2, 500), "Pune": (1, 500)}

    for fn in (summarize_with_sql, summarize_with_pandas):
        result = fn(orders).set_index("city")
        for city, (count, revenue) in expected.items():
            assert int(result.loc[city, "order_count"]) == count
            assert int(result.loc[city, "total_revenue"]) == revenue
    print("All GROUP BY checks passed.")

```

15. Code Walkthrough

- `df.to_sql("orders", conn, ...)` loads the DataFrame into a real, queryable SQLite table so we can run genuine SQL against it.
- **The SQL string** is the star: `GROUP BY city` forms the piles, `COUNT(*)` and `SUM(amount)` compute the per-pile numbers, and `HAVING COUNT(*) >= ?` filters piles *after* they exist. The `?` placeholder is a parameterized query — never build SQL with string concatenation, as that invites SQL injection.
- `summarize_with_pandas` mirrors the SQL exactly. `.groupby("city")` is `GROUP BY`, `.agg(...)` supplies the aggregate functions, and the boolean mask `grouped["order_count"] >= min_orders` plays the role of `HAVING`. Notice the filter happens *after* `.agg()` — you can't filter on a count that hasn't been computed yet, in Python just as in SQL.
- **The `__main__` block** is a small self-check: it asserts both implementations produce identical, correct numbers. If either GROUP BY breaks, the script fails loudly instead of silently returning wrong data.

16. Dry Run

Let's trace `summarize_with_sql(orders, min_orders=2)` end to end.

1. **FROM / load:** Six rows enter the pipeline.
2. **WHERE:** No `WHERE` clause here, so all six rows survive.
3. **GROUP BY city:** Rows sort into three piles.
4. Delhi → rows 1, 3, 5
5. Mumbai → rows 2, 4
6. Pune → row 6
7. **Aggregate:** Compute per pile.
8. Delhi → `COUNT = 3` , `SUM = 700`
9. Mumbai → `COUNT = 2` , `SUM = 500`
10. Pune → `COUNT = 1` , `SUM = 500`
11. **HAVING COUNT(*) >= 2:** Pune has a count of 1, so its pile is discarded. Delhi and Mumbai stay.
12. **ORDER BY total_revenue DESC:** Delhi (700) comes before Mumbai (500).

Final result:

city	order_count	total_revenue
Delhi	3	700
Mumbai	2	500

Pune vanished not because of a bad row, but because its *finished pile* failed the `HAVING` test — the perfect illustration of why `HAVING` runs last.

17. Complexity Analysis

Time complexity: $O(n)$ on average (or $O(n \log n)$ if grouping requires a sort). The database makes a single pass over the n rows, dropping each into its bucket — typically via a hash on the grouping key. Compare that to the brute-force $O(k \times n)$ of one query per group. One scan beats many.

Space complexity: $O(g)$, where g is the number of distinct groups. The engine holds one accumulator per group (a running count and sum). If every row were unique in the grouping column, g approaches n in the worst case, but for real reports g is tiny — five cities, twelve months, a handful of categories.

The reason this is optimal: you fundamentally must look at every row at least once to summarize it, so you can't beat $O(n)$ on the read. `GROUP BY` hits that lower bound in a single pass.

18. Alternative Solutions

GROUP BY isn't always the right tool. Here's how it compares:

Approach	What it does	When to use
GROUP BY	Collapses rows into one summary per group	You want a report: one row per category, detail lost
Window functions	Adds a running total or per-group average <i>while keeping every original row</i>	You need both the detail and the summary
DISTINCT	Removes duplicate rows only	You just want unique values, no summarizing

A window function example that keeps every row while adding a city total:

```
SELECT order_id,
       city,
       amount,
       SUM(amount) OVER (PARTITION BY city) AS city_total
FROM orders;
```

Here you still see all six orders, but each row now carries its city's total alongside it. Reach for **GROUP BY** when you genuinely want one row per category and are happy to lose the individual rows.

19. Edge Cases

- **Empty table:** `SELECT COUNT(*) FROM orders` returns `0` (one row). But `SELECT city, COUNT(*) FROM orders GROUP BY city` returns *zero rows* — no rows means no piles.
- **NULL in the grouping column:** All `NULL` s collect into a single group of their own. Don't assume they vanish.
- **SUM / AVG ignoring NULL :** Aggregates skip `NULL` values. `AVG` divides by the count of *non-null* values, which can surprise you if you expected `NULL` treated as zero.
- **Every row unique:** Grouping by a primary key gives you one group per row — technically valid, but pointless, since nothing collapses.
- **Aggregate with no GROUP BY :** `SELECT city, COUNT(*) FROM orders` (no `GROUP BY`) is either an error or silently collapses everything into a single row — rarely what you meant.

20. Common Mistakes

1. **Putting an aggregate in WHERE .** `WHERE COUNT(*) > 100` always errors, because `WHERE` runs *before* any count is computed. Move it to `HAVING` . This is the classic bug — the count simply doesn't exist yet when `WHERE` runs.

2. **Selecting an ungrouped, unaggregated column.** `SELECT city, customer_name ... GROUP BY city` breaks, because each city pile holds many names. Group by it or aggregate it.
3. **Using an aggregate but forgetting `GROUP BY`.** This quietly collapses the whole table into one row when you wanted one row per category — a silent logic bug, not an error.
4. **Filtering raw values in `HAVING` when `WHERE` would do.** Both may work, but `HAVING` builds every pile *first* and then throws most away. On a million-row table, that's the difference between instant and painful. **Rule of thumb: raw values go in `WHERE`, group totals go in `HAVING`.**
5. **Assuming aggregates count `NULL` s.** `COUNT(column)` skips `NULL`, while `COUNT(*)` counts rows regardless. Mixing these up produces subtly wrong totals.

21. Interview Questions

- What's the difference between `WHERE` and `HAVING`, and why can't you swap them?
- In what order does SQL logically execute the clauses of a query?
- Why must every non-aggregated column in `SELECT` appear in `GROUP BY`?
- What's the difference between `COUNT(*)` and `COUNT(column)`?
- Given a slow report, how would you decide whether a filter belongs in `WHERE` or `HAVING`?
- When would you choose a window function over `GROUP BY`?
- How does `GROUP BY` treat `NULL` values in the grouping column?

22. Similar Problems

While `GROUP BY` is a SQL construct, the *aggregation-by-key* pattern is everywhere in coding interviews and data structures work:

- **LeetCode 1741 — Find Total Time Spent by Each Employee:** direct `GROUP BY` and `SUM` practice.
- **LeetCode 1050 — Actors and Directors Who Cooperated At Least Three Times:** `GROUP BY` plus `HAVING COUNT(*) >= 3`.
- **LeetCode 596 — Classes More Than 5 Students:** the canonical `HAVING` filter on a count.
- **LeetCode 1 — Two Sum:** not SQL, but the same "hash rows into buckets by a key" idea that powers how databases group internally.

They're related because all of them reduce many records to a per-key summary — the exact mental model behind aggregate functions and `GROUP BY`.

23. Key Takeaways

- **Aggregate functions** (`COUNT` , `SUM` , `AVG`) crush many rows into one number.
- **GROUP BY** decides which rows share a pile, then runs the aggregate on each pile.
- **WHERE filters rows *before* grouping; HAVING filters piles *after*.** Never put an aggregate in `WHERE` .
- **The golden rule:** every `SELECT` column must be grouped or aggregated.
- **Performance:** push raw-value filters into `WHERE` so you build fewer piles from the start.

Keep asking "*does this value exist yet at this stage?*" and GROUP BY stops being confusing.

24. Watch the Video

Reading builds the map; watching the animation makes it move. See the rows physically drift into their colored piles and collapse into summary rows — it's the moment GROUP BY finally clicks. **Watch Lesson 8 here: {LINK}**

25. About the Series

This is Lesson 8 of the **Fun with Learning Technology** series — a hands-on journey through SQL and software engineering that trades dry theory for real intuition. Each lesson takes one concept most tutorials rush past, breaks it down with plain-language mental models, and shows you the mistakes to dodge before they cost you an interview or a production bug. Whether you're learning your first `SELECT` or sharpening for a data-engineering role, the series meets you where you are and levels you up one clear idea at a time.

26. Call To Action

If this made GROUP BY click, **like the video on YouTube** — it genuinely helps the channel reach more developers. **Subscribe** so you don't miss the next lesson on window functions, and **subscribe to the Substack** for the written deep-dives like this one. Got a query that stumped you? **Drop it in the comments** — real questions become future lessons. And if a teammate keeps writing `WHERE COUNT(*)` , **share this with them**. A like and a follow take none of your time and mean a lot.

Quick Review

GROUP BY & HAVING: when do you reach for them?

When a question needs per-category summaries or filtering by a computed metric ('per', 'each', 'average', 'count of'). Tell: aggregate + a grouping attribute.

Time complexity of a GROUP BY aggregation?

$O(n)$ with a hash aggregate over n rows; $O(n \log n)$ if the engine sorts rows to group them. Each row is visited once.

Space complexity of GROUP BY?

$O(g)$ for g distinct groups held in the hash/aggregate table. Worst case $O(n)$ when every row is its own group.

Common bug: filtering an aggregate in WHERE?

WHERE runs before grouping, so it can't see aggregates. Filter computed metrics like $COUNT(*) > 5$ in HAVING; filter raw rows in WHERE.

WHERE vs HAVING?

WHERE filters individual rows before grouping; HAVING filters whole groups after aggregation. Use WHERE to shrink input, HAVING to test group results.

Interview: non-aggregated column not in GROUP BY?

Illegal in standard SQL — every selected column must be grouped or aggregated. MySQL may allow it but returns an arbitrary row's value.

sql Lesson 8: GROUP BY and HAVING (GROUP BY and HAVING are SQL clauses used to transform raw, individual rows of data into summarized, meaningful insights. These tools allow you to categorize data based on shared attributes and filter those results based on calculated metrics rather than raw values. You reach for these when you need to answer analytical questions, such as identifying trends or finding outliers within specific segments of your database.) — free Python cheat sheet